

P.PORTO

**ESCOLA
SUPERIOR
DE TECNOLOGIA
E GESTÃO**

SUPERMARKET MANAGEMENT PLATFORM

PAW



Elaborado por:

Regina da Paz- 8210253

Danilo Ramos-

Índice

Introdução	3
1. Contextualização	4
2. Arquitetura do Sistema.....	5
2.1. Padrão MVC:	5
3. Requisitos funcionais:.....	6
4. Requisitos não funcionais:	7
5. Diagrama de classes	7
6. Backoffice	9
6.1. Instruções	9
7. API.....	10
7.1. Controladores e Rotas	10
7.2. Swagger.....	10
8. Frontoffice	11
8.1. Serviços	11
8.3. Dados.....	12
9. Conclusão	13

Índice figuras

Figura 1: Cluster do MongoDB	5
Figura 2: Diagrama de classes Supermarket	8

Introdução

No âmbito da unidade curricular de Programação em Ambientes Web (PAW), foi proposto o desenvolvimento de uma plataforma web orientada à gestão de supermercados e encomendas online. O principal objetivo deste projeto consiste na criação de um sistema capaz de simular o funcionamento de um ambiente real de comércio digital, integrando funcionalidades de gestão de produtos, supermercados, vendas presenciais e encomendas com entrega.

A aplicação desenvolvida pretende responder às necessidades de diferentes tipos de utilizadores, nomeadamente administradores, supermercados, clientes e estafetas, disponibilizando funcionalidades específicas para cada perfil. Para além da gestão de produtos e supermercados, o sistema permite o registo de vendas em caixa, processamento de encomendas, definição de métodos de entrega e acompanhamento do estado das encomendas.

O desenvolvimento do projeto foi realizado utilizando a framework ExpressJS para Node.js, juntamente com o template engine EJS para a construção das interfaces web e MongoDB para a persistência de dados. Adicionalmente, a aplicação começou a ser preparada para uma arquitetura baseada em APIs REST, permitindo futura integração com frontend desenvolvido em Angular.

Ao longo do desenvolvimento foram aplicados conhecimentos relacionados com desenvolvimento web full-stack, modelação de dados, arquitetura MVC, gestão de rotas, autenticação de utilizadores e implementação de regras de negócio, consolidando competências fundamentais no contexto do desenvolvimento moderno de aplicações web.

1. Contextualização

O crescimento acelerado do comércio eletrónico e da digitalização dos serviços de entrega transformou significativamente o setor da distribuição alimentar. Atualmente, os consumidores procuram soluções digitais que lhes permitam realizar compras online de forma rápida, segura e conveniente, sem necessidade de deslocação física aos supermercados.

Neste contexto, os supermercados passaram a investir em plataformas digitais capazes de gerir não apenas a venda de produtos, mas também toda a logística associada às encomendas e entregas. A gestão de stock, processamento de encomendas, métodos de entrega e acompanhamento em tempo real tornaram-se elementos essenciais para garantir eficiência operacional e satisfação do cliente.

O projeto desenvolvido pretende simular uma plataforma completa de supermercado online, contemplando tanto a componente administrativa como a experiência do consumidor final. A aplicação permite aos clientes consultar supermercados e produtos disponíveis, realizar encomendas e selecionar diferentes métodos de entrega, incluindo levantamento em loja ou entrega por estafeta. Paralelamente, os supermercados podem gerir os seus produtos e encomendas, enquanto os estafetas possuem funcionalidades relacionadas com a aceitação e realização de entregas.

A solução contempla ainda regras de negócio realistas, como:

- aprovação de supermercados por administradores;
- associação de produtos a supermercados específicos;
- gestão de estados de encomenda;
- cálculo automático de custos de entrega;
- atribuição de estafetas a encomendas;
- controlo de stock e processamento de vendas.

A arquitetura da aplicação foi desenvolvida com base numa separação clara entre frontend e backend, recorrendo a Angular para a interface do utilizador e ExpressJS com MongoDB para a camada de serviços e persistência de dados. Esta abordagem permite maior flexibilidade, reutilização de APIs e preparação para futuras expansões da plataforma.

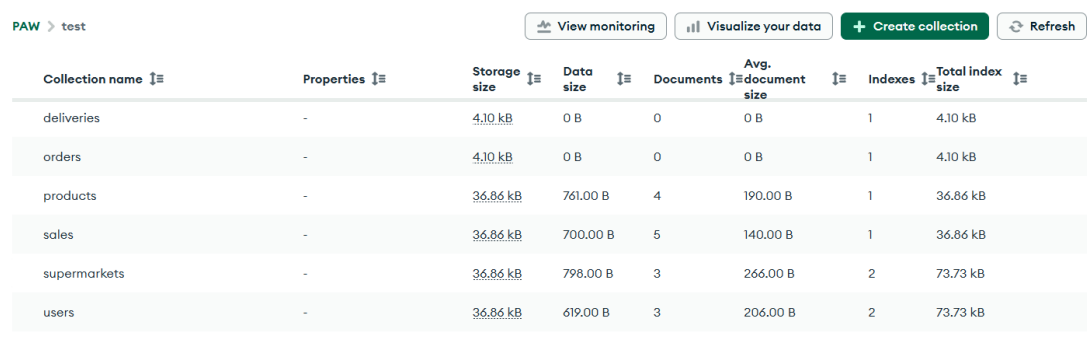
2. Arquitetura do Sistema

A aplicação foi desenvolvida segundo uma arquitetura modular baseada na separação entre frontend e backend, permitindo maior organização, reutilização de código e facilidade de manutenção.

O frontend foi desenvolvido utilizando angular, responsável pela interface gráfica e interação com o utilizador. Esta componente comunica com o backend através de serviços REST, permitindo o carregamento e manipulação dinâmica dos dados da aplicação.

O backend foi desenvolvido com Node.js e ExpressJS, sendo responsável pelo processamento da lógica de negócio, autenticação, validação de dados e comunicação com a base de dados MongoDB. Para garantir melhor organização e separação de responsabilidades, foi utilizada uma arquitetura baseada no padrão MVC (Model-View-Controller).

A persistência de dados é assegurada pelo MongoDB, permitindo armazenar informações relacionadas com utilizadores, supermercados, produtos, encomendas e entregas.



Collection name	Properties	Storage size	Data size	Documents	Avg. document size	Indexes	Total index size
deliveries	-	4.10 kB	0 B	0	0 B	1	4.10 kB
orders	-	4.10 kB	0 B	0	0 B	1	4.10 kB
products	-	36.86 kB	761.00 B	4	190.00 B	1	36.86 kB
sales	-	36.86 kB	700.00 B	5	140.00 B	1	36.86 kB
supermarkets	-	36.86 kB	798.00 B	3	266.00 B	2	73.73 kB
users	-	36.86 kB	619.00 B	3	206.00 B	2	73.73 kB

Figura 1: Base de dados do MongoDB

A comunicação entre frontend e backend é realizada através de APIs REST documentadas com Swagger, permitindo uma estrutura organizada, reutilizável e preparada para futuras integrações.

2.1. Padrão MVC:

O backend da aplicação foi desenvolvido com base no padrão arquitetural MVC (Model-View-Controller), permitindo uma separação clara entre os diferentes componentes do sistema.

- Models: responsáveis pela estruturação e manipulação dos dados armazenados na base de dados;
- Views: responsáveis pela apresentação visual da informação ao utilizador;
- Controllers: responsáveis pelo processamento da lógica de negócio e gestão das requisições.

A utilização deste padrão permitiu melhorar a organização do código, facilitar a manutenção da aplicação e promover maior reutilização dos componentes desenvolvidos

3. Requisitos funcionais:

A plataforma desenvolvida contempla um conjunto de funcionalidades principais definidas de acordo com o enunciado do projeto.

Entre os principais requisitos funcionais implementados destacam-se:

- Registo e autenticação de utilizadores;
- Gestão de supermercados;
- Gestão de produtos e categorias;
- Gestão de encomendas;
- Gestão de carrinho de compras;
- Comparação de preços entre supermercados;
- Gestão de entregas;
- Gestão de estados das encomendas;
- Dashboards administrativos;
- APIs REST para comunicação frontend/backend;
- Documentação Swagger.

4. Requisitos não funcionais:

Para além das funcionalidades principais, foram considerados diversos requisitos não funcionais importantes para garantir qualidade e eficiência da aplicação.

Destacam-se:

- Interface responsiva;
- Organização modular da aplicação;
- Facilidade de manutenção e escalabilidade;
- Segurança na autenticação e acesso aos dados;
- Separação entre frontend e backend;
- Estrutura preparada para futuras expansões;
- Boa experiência de utilização;

5. Diagrama de classes

Os diagramas de classe são fundamentais no desenvolvimento de software orientado a objetos, oferecendo uma representação visual da estrutura do sistema. No nosso projeto de Programação em Ambiente Web, utilizamos estes diagramas para organizar e definir claramente as entidades e suas inter-relações, facilitando a compreensão e o planeamento da arquitetura do sistema.

Estes diagramas ilustram as classes, e os relacionamentos entre elas, proporcionando uma visão detalhada das interconexões e colaborações dentro do sistema. Ao servir como um guia para a codificação e uma ferramenta de comunicação eficaz entre os membros da equipe, os diagramas de classe garantiram que todos compartilhassem uma compreensão comum da estrutura do software.

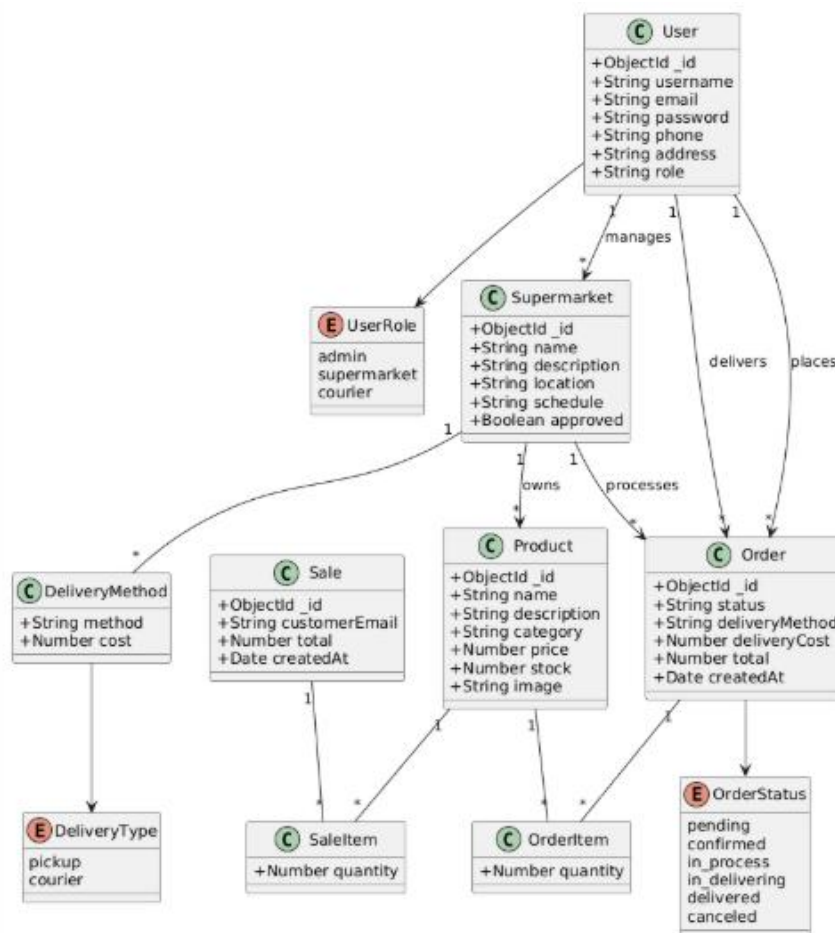


Figura 2: Diagrama de classes Supermarket

A classe **User** representa os utilizadores da aplicação, armazenando informações como nome, email, password, telefone, morada e perfil de acesso. Os utilizadores podem assumir diferentes papéis definidos na enumeração **UserRole**, nomeadamente administrador, supermercado ou estafeta.

A classe **Supermarket** representa os supermercados registados na plataforma, contendo dados como nome, descrição, localização, horário de funcionamento e estado de aprovação. Cada supermercado é gerido por um utilizador e possui vários produtos e métodos de entrega.

Os produtos são representados pela classe **Product**, que armazena informações como nome, descrição, categoria, preço, stock e imagem. Cada produto pertence a um supermercado específico.

A classe **Order** representa as encomendas realizadas pelos clientes, contendo informações relativas ao método de entrega, custo de entrega, valor total e data de criação. Cada encomenda pode possuir vários produtos através da entidade

intermédia **OrderItem**, responsável por armazenar a quantidade de cada produto incluído na encomenda.

O sistema inclui ainda a classe **Sale**, utilizada para representar vendas presenciais em caixa, associando produtos através da entidade **SaleItem**.

Os métodos de entrega são representados pela classe **DeliveryMethod**, sendo os tipos de entrega definidos na enumeração **DeliveryType**, que inclui levantamento em loja e entrega por estafeta.

Por fim, o estado das encomendas é controlado pela enumeração **OrderStatus**, permitindo acompanhar o ciclo de vida da encomenda desde o estado pendente até entregue ou cancelada.

6. Backoffice

6.1. Instruções

O backoffice da aplicação foi desenvolvido com o objetivo de permitir a administração e gestão das principais entidades do sistema, nomeadamente supermercados, produtos, vendas, encomendas e utilizadores. Esta componente destina-se principalmente a administradores e supermercados, fornecendo ferramentas necessárias para a gestão operacional da plataforma.

Para além da implementação técnica, foi dada especial atenção à organização do projeto e à documentação interna do sistema, de forma a facilitar futuras evoluções e permitir que novos desenvolvedores possam integrar-se rapidamente no desenvolvimento da aplicação. Num projeto colaborativo e escalável, a existência de instruções claras e de uma estrutura organizada é fundamental para garantir manutenção eficiente, redução de erros e continuidade do desenvolvimento ao longo do tempo.

Assim, procurou-se manter uma arquitetura modular, nomenclatura consistente e separação clara entre componentes, permitindo que futuras equipas consigam compreender facilmente a lógica da aplicação, reutilizar componentes existentes e adicionar novas funcionalidades sem comprometer a estabilidade do sistema.

A aplicação encontra-se organizada segundo uma arquitetura baseada em Node.js e ExpressJS, utilizando EJS para renderização das interfaces administrativas e MongoDB para persistência de dados.

Requisitos

Para execução do projeto são necessários os seguintes componentes:

- Node.js;
- NPM;
- MongoDB Atlas ou MongoDB local;
- Dependências especificadas no ficheiro package.json.

Instalação das dependências:

- `npm install`

Execução da aplicação:

- `node app.js`

Por fim no browser, na barra de navegação inserir “localhost:3000”.

7. API

7.1. Controladores e Rotas

Na construção da API REST, foram desenvolvidos controladores dedicados para cada serviço. Para manter a organização e facilitar a compreensão, as rotas de cada controlador iniciam com o nome do serviço em sua forma pluralizada.

7.2. Swagger

O Swagger foi utilizado para documentar todos os serviços REST disponibilizados pela aplicação, permitindo uma visualização clara e organizada dos endpoints desenvolvidos. Esta ferramenta facilita a compreensão da API, simplifica os processos de teste e validação dos serviços e constitui uma importante fonte de documentação para futuros desenvolvedores.

A documentação gerada apresenta, para cada endpoint, o respetivo método HTTP, descrição da funcionalidade, parâmetros de entrada, estrutura dos pedidos e respostas, bem como os possíveis códigos de retorno. Desta forma, torna-se possível testar diretamente os serviços através da interface disponibilizada pelo Swagger UI, sem necessidade de recorrer a ferramentas externas.

A utilização desta tecnologia permitiu ainda garantir uma maior consistência na definição dos contratos entre frontend e backend, facilitando a integração entre as diferentes componentes da aplicação.

Principais grupos de endpoints documentados:

- Autenticação e gestão de utilizadores;

- Gestão de supermercados;
- Gestão de produtos;
- Gestão de encomendas;
- Gestão de entregas;
- Gestão de vendas;
- Gestão de dashboards e estatísticas.

8. Frontoffice

O Frontoffice corresponde à componente visível e interativa da plataforma, desenvolvida em Angular, permitindo a interação entre os diferentes tipos de utilizadores e o sistema. Esta camada foi concebida com foco na usabilidade, organização modular e integração com os serviços disponibilizados pelo backend.

A interface foi estruturada de forma a oferecer uma experiência moderna e intuitiva.

O sistema contempla diferentes tipos de utilizadores:

- Cliente;
- Supermercado;
- Estafeta (Courier);
- Administrador.

Cada utilizador possui permissões, funcionalidades e dashboards específicos, permitindo uma utilização segmentada da plataforma.

8.1. Serviços

A comunicação entre o frontend e o backend é realizada através de serviços Angular, responsáveis pela gestão das operações HTTP e pelo consumo da API REST.

Os principais serviços implementados foram:

Serviço	Responsabilidade
AuthService	Autenticação e gestão de sessões
UserService	Gestão de utilizadores
SupermarketService	Gestão de supermercados
ProductService	Gestão de produtos
OrderService	Gestão de encomendas
DeliveryService	Gestão de entregas
DashboardService	Obtenção de dados estatísticos

8.3. Dados

A gestão de dados da aplicação é realizada através da integração entre os serviços Angular e os endpoints disponibilizados pela API REST.

Os dados são obtidos dinamicamente através de pedidos HTTP, permitindo que a informação apresentada ao utilizador esteja permanentemente sincronizada com a base de dados MongoDB.

Para garantir uma melhor organização do código, foram utilizadas interfaces TypeScript para representar as principais entidades do sistema, incluindo:

- User;
- Product;
- Supermarket;
- Order;
- Delivery;
- Sale.

Esta abordagem aumenta a legibilidade do código e reduz a probabilidade de erros durante o desenvolvimento

9. Conclusão

A realização deste projeto permitiu aplicar de forma prática os conhecimentos adquiridos ao longo da unidade curricular de Programação em Ambientes Web, consolidando competências relacionadas com desenvolvimento full-stack, arquitetura de aplicações web, APIs REST e modelação de dados.

A plataforma desenvolvida conseguiu responder aos principais objetivos inicialmente definidos, disponibilizando funcionalidades de gestão de supermercados, produtos, encomendas e entregas, integrando diferentes tipos de utilizadores numa solução única e centralizada.

Ao longo do desenvolvimento foram enfrentados diversos desafios técnicos, principalmente relacionados com a integração entre frontend e backend, organização da arquitetura da aplicação e implementação das regras de negócio. No entanto, essas dificuldades contribuíram significativamente para o aprofundamento dos conhecimentos técnicos e para o desenvolvimento de capacidades de resolução de problemas.

De forma geral, o projeto representou uma experiência importante de desenvolvimento num contexto real, permitindo compreender melhor os desafios envolvidos no desenvolvimento de aplicações web modernas.